# Nesne Tabanlı Programlama

Dr. Öğr. Üyesi Emre Bendeş

# Final Sözcüğü

- Bir değişkende ;
  - Değerinin değiştirilemeyeceğini söyler (Sabit tanımı)
  - Mutlaka ilk değer verilmelidir.

```
final int a = 0;
```

- Bir fonksiyonda;
  - Fonksiyonun override olamayacağını belirtir

```
final void deneme(){
        // bir şeyler yap
}
```

- Bir sınıfta;
  - Sınıfın miras alınamayacağını belirtir.

```
public final class Ders {
……
}
```

# İç Sınıflar

- Bir sınıfın içerisinde sınıf tanımlanabilir.
  - Static ya da non-static olabilir.
  - İçi sınıf, dış sınıfın bir üyesi olduğundan **public, private, package-private** ya da **protected** olabilir.

    HATIRLATMA: dış sınıf sadece **public** ya da **package-private** olabilir

```
class OuterClass {
    ...
    class InnerClass {
        ...
    }
    static class StaticNestedClass {
        ...
    }
}
```

# İç Sınıflar

- **Yalnızca bir yerde kullanılan sınıfları mantıksal olarak gruplamanın bir yoludur** :
  - Bir sınıf yalnızca bir başka sınıf için faydalıysa, o sınıfa gömmek ve ikisini bir arada tutmak mantıklıdır. Bu tür "yardımcı sınıfları" iç içe yerleştirmek, paketlerini daha akıcı hale getirir.
- **Kapsüllemeyi artırır.**

Ya da

B nin kendide private olabilir

- **Daha okunabilir ve bakımı yapılabilir kodlar yazılabilir**:
  - Küçük sınıfları üst düzey sınıflara yerleştirmek, kodu kullanıldığı yere daha yakın hale getirir.

# non-static iç sınıf

- Bir iç sınıf, kendisini çevreleyen sınıftan türetilen nesne ile ilişkilendirilir ve bu nesnenin yöntemlerine ve alanlarına doğrudan erişime sahiptir.
- Bir iç sınıf bir nesne ile ilişkilendirildiğinden, kendi içinde herhangi bir **statik üye tanımlayamaz**.
- Bir iç sınıf nesnesi yalnızca bir dış sınıf nesnesinin içinde var olabilir

```
OuterClass outerObject = new OuterClass();
OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

```
class OuterClass {
    ...
    class InnerClass {
        ...
    }
    static class StaticNestedClass {
        ...
    }
}
```

# static iç sınıf

- Sınıf yöntemleri ve değişkenlerinde olduğu gibi, statik iç sınıf, dış sınıfıyla (nesne ile değil) ilişkilendirilir.
- Statik sınıf yöntemleri gibi, bir statik iç sınıf, doğrudan dış sınıfında tanımlanan örnek değişkenlere veya yöntemlere başvuramaz
  - bunları yalnızca bir nesne referansı aracılığıyla kullanabilir.

```
public class OuterClass {
    String outerField = "Outer field";
    static String staticOuterField = "Static outer field";

    static class StaticNestedClass {
        void accessMembers(OuterClass outer) {
            System.out.println(outerField); //DERLEME HATASI VERİR !!!!!!!
            System.out.println(outer.outerField);
            System.out.println(staticOuterField);
        }
    }
}
```

- Static iç sınıftan nesne normal sınıf olduğu gibi üretilir.

```
StaticNestedClass staticNestedObject = new StaticNestedClass();
```

- Temelde, mantıksal olarak, hiyerarşi içerisinde uygun paketleme sağlar.

# Yerel Sınıflar

- Bir iç sınıftır.
- Sınıfın içerisindeki bir blokta ({.....})tanımlanmıştır. Örneğin bir metot içinde.
- Dış sınıfın üyelerine erişebilir.
- Yerel değişkenlere erişebilir
  - JDK 8 öncesi için lokal değişken **final** tanımlanmalı
  - JDK 8 sonrası için **efektif final** olmalı (ilk değer verildikten sonra hiç değiştirilmemeli)
- Metot paramerelerine erişebilir
  - JDK 8 ve sonrası için
- Yerel sınıflarda static tanımlama yapılamaz.
  - Yerel sınıflar, çevreleyen bloğun nesne üyelerine erişimleri olduğu için statik değildir.
  - Static final (sabit) tanımı yapılabilir
- İçerisinde interface tanımlanamaz.
  - interface varsayılan static dir
  - interface metot içinde de tanımlanamaz.

```java
public class LocalClassExample {
    static String regularExpression = "[^0-9]";
    public static void validatePhoneNumber(
        String phoneNumber1, String phoneNumber2) {
        final int numberLength = 10;
        class PhoneNumber {
            String formattedPhoneNumber = null;
            PhoneNumber(String phoneNumber){
                // numberLength = 7;
                String currentNumber = phoneNumber.replaceAll(
                  regularExpression, "");
                if (currentNumber.length() == numberLength)
                    formattedPhoneNumber = currentNumber;
                else
                    formattedPhoneNumber = null;
            }
            public String getNumber() {
                return formattedPhoneNumber;
            }
        }
        PhoneNumber myNumber1 = new PhoneNumber(phoneNumber1);
        PhoneNumber myNumber2 = new PhoneNumber(phoneNumber2);

        if (myNumber1.getNumber() == null)
            System.out.println("First number is invalid");
        else
            System.out.println("First number is " + myNumber1.getNumber());
        if (myNumber2.getNumber() == null)
            System.out.println("Second number is invalid");
        else
            System.out.println("Second number is " + myNumber2.getNumber());
    }
```

```java
    public static void main(String... args) {
            validatePhoneNumber("123-456-7890", "456-7890");
    }
```

# Shadowing

```java
public class ShadowTest {

    public int x = 0;

    class FirstLevel {

        public int x = 1;

        void methodInFirstLevel(int x) {
            System.out.println("x = " + x);
            System.out.println("this.x = " + this.x);
            System.out.println("ShadowTest.this.x = " + ShadowTest.this.x);
        }
    }

    public static void main(String... args) {
        ShadowTest st = new ShadowTest();
        ShadowTest.FirstLevel fl = st.new FirstLevel();
        fl.methodInFirstLevel(23);
    }
}
```

Ekran Çıktısı :

```
x = 23
this.x = 1

ShadowTest.this.x = 0
```

# ENUM Türü

- Bir *enum türü,* bir değişkeni, değeri önceden tanımlanmış bir dizi sabitler olabilmesini sağlayan özel bir veri türüdür.
- Örneğin, KUZEY GÜNEY DOĞU BATI

```
public enum Day {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY
}
```

```
public static void main(String[] args) {
    EnumTest firstDay = new EnumTest(Day.MONDAY);
    firstDay.tellItLikeItIs();
}
```

```
public class EnumTest {
    Day day;
    public EnumTest(Day day) {
        this.day = day;
    }
    public void tellItLikeItIs() {
        switch (day) {
            case MONDAY:
                System.out.println("Mondays are bad.");
                break;
            case FRIDAY:
                System.out.println("Fridays are better.");
                break;
            case SATURDAY: case SUNDAY:
                System.out.println("Weekends are best.");
                break;
            default:
                System.out.println("Midweek days are so-so.");
                break;
        }
    }
```

# MİRAS

- Bir sınıfın diğer bir sınıf özelliğini alması

```
class Y {...}
class X extends Y {...}
```

X sınıfı Y sınıfını miras alır.

X sınıfından türeyen nesne Y sınıfının da değişken ve metotlarını içerir

- Hiyerarşik miras alma durumu olabilir

```
class Z {...}
class Y extends Z {...}
class X extends Y {...}
```

Y sınıfı Z sınıfını miras alır. X sınıfı Y ve Z sınıfını miras alır.

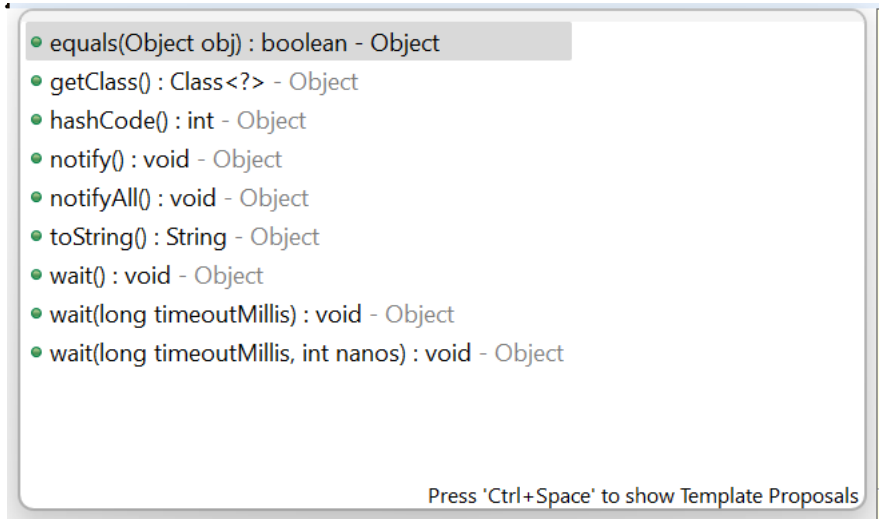
# VARSAYILAN MİRAS

- Java da her sınıf Object sınıfını miras alır. (java.lang paketinde)

(eclipse'de ctrl+shift+t ile open type penceresinden aranarak kaynak kodları görüntülenebilir)

```
public class X {
      // İçi Boş bir sınıf
}

X x=new X();
x.
```



# Overriding – Hiding

## Override

```
public class Ogrenci {
  public Ogrenci() {
         dersSec();
  }
  public void dersSec() {
     System.out.println("Ogrenci Sınıfı içi");
  }
}
```

```
public class LisansOgrencisi extends Ogrenci {
   public LisansOgrencisi() {
         dersSec();
   }
   public void dersSec() {
     System.out.println("Lisansogrenci Sınıfı içi");
   }
}
```

## Hide

```
public class Ogrenci {
  public Ogrenci() {
         dersSec();
  }
  public static void dersSec() {
     System.out.println("Ogrenci Sınıfı içi");
  }
}
```

```
public class LisansOgrencisi extends Ogrenci {
   public LisansOgrencisi() {
         dersSec();
   }
   public static void dersSec() {
     System.out.println("Lisansogrenci Sınıfı içi");
   }
}
```

```
LisansOgrencisi ali = new LisansOgrencisi();
```

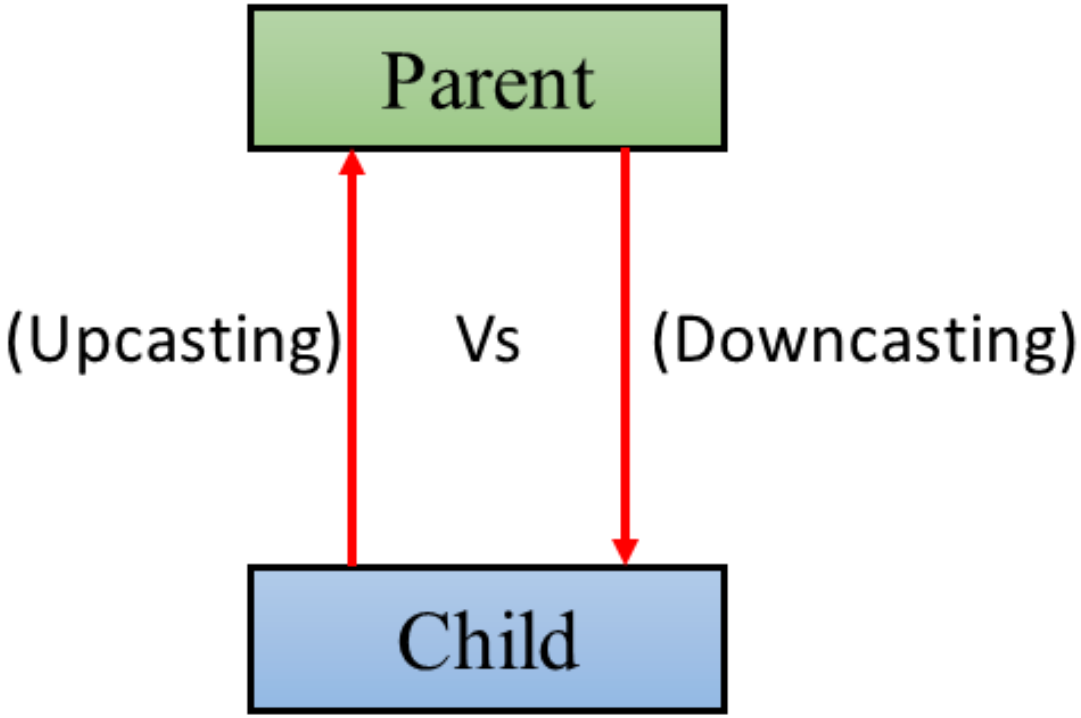
```
LisansOgrencisi Sınıfı içi
LisansOgrencisi Sınıfı içi
```

```
Ogrenci Sınıfı içi
LisansOgrencisi Sınıfı içi
```

# Sınıflar Arası Dönüşüm (casting)

- Birbirine **miras** yolu ile ile bağlı nesneler arası tür dönüşümü yapılabilir.

Altsınıf nesnesinin üst sınıf değişkenine atanması




Üst sınıf nesnesinin alt sınıf değişkenine atanması

# Upcasting – Downcasting

```
public class Ogrenci {
  String Ad;
  String Soyad;
  public Ogrenci(String ad, String soyad) {
     Ad = ad;
     Soyad = soyad;
  }
  public void dersSec() {
     System.out.println(this.ad+«öğrencisi");
   }
}
```

```
public class LisansOgrencisi extends Ogrenci
{
  public void stajYap() {
     System.out.println("staj yapılıyor…");
  }
}
```

```
public class YLOgrencisi extends Ogrenci{
  public void tezYap() {
    System.out.println("tez yapılıyor…");
  }
}
```

```
public class Ders {
  ArrayList<Ogrenci> liste = new ArrayList<>();
  public void dersSec(Ogrenci ogr) {
    liste.add(ogr);
    if (ogr instanceof LisansOgrencisi) {
      LisansOgrencisi ogrenci = (LisansOgrencisi) ogr;
      ogrenci.stajYap();
    }
  }
}
```

```
LisansOgrencisi ali = new LisansOgrencisi();
Ders ders = new Ders();
ders.dersSec(ali);
```

**DOWNCASTING**

**UPCASTING**

# Ust sınıftaki metotla aynı imzaya sahip metot tanımlandığında ne olur?

| | Superclass Instance Method | Superclass Static Method |
|---|---|---|
| Subclass Instance Method | Overrides | Generates a compile-time error |
| Subclass Static Method | Generates a compile-time error | Hides |

# Erişim Belirleyiciler

Sınıf seviyesinde- Nerede sınıfdan nesne üretbilir

Üye seviyesinde -Diğer sınıfların belirli bir alanı (değişken) kullanıp kullanamayacağını veya belirli bir metodu çağırıp çağıramayacağını belirler.

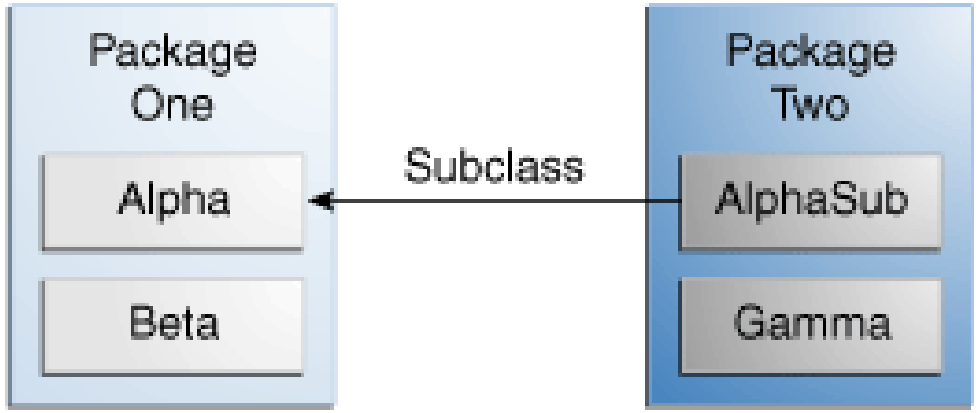
| Üst düzey (sınıf seviyesinde) | Üye seviyesinde |
| --- | --- |
| • Public<br>• Package-private (boş) | • Public<br>• Private<br>• Protected<br>• Package-private (boş) |

# Erişim Belirleyiciler

Bir üyeye nerelerden ulaşılabilir:

| Belirleyici | Class | Package | Subclass | Dünya |
|---|---|---|---|---|
| Public | E | E | E | E |
| protected | E | E | E | H |
| *Belirleyici yok* | E | E | H | H |
| private | E | H | H | H |

# Erişim Belirleyiciler




## Alpha sınıfı üyelerine nerelerden erişilebilir ?

| Belirleyici | Alpha | Beta | Alphasub | Gamma |
|---|---|---|---|---|
| public | E | E | E | E |
| protected | E | E | E | H |
| *Belirleyici yok* | E | E | H | H |
| private | E | H | H | H |

# Erişim Belirleyiciler

- Diğer programcılar sınıfınızı kullanıyorsa:
  - Yanlış kullanımdan kaynaklanan hataların olmayacağından emin olmak istersiniz.
  - Erişim seviyeleri bunu yapmanıza yardımcı olabilir.
  - Belirli bir üye için anlamlı olan en kısıtlayıcı erişim düzeyini kullanın.
  - Sabitler dışındaki alanlarda *public* kullanımından kaçının.
  - Private kullanmak için geçerli bir neden yoksa kullanın
    - Erişimi **getter** ve **setter** metotları ile kontrollü bir şekilde sağlayın.

# Interface

- Yazılım mühendisliğinde, farklı programcı gruplarının yazılımlarının nasıl etkileşime girdiğini açıklayan bir "sözleşme" üzerinde anlaşmaya varmasının önemli olduğu bazı durumlar vardır.
- Her grup, diğer grubun kodunun nasıl yazıldığını bilmeden kendi kodunu yazabilmelidir.
- Genel olarak konuşursak, *arayüzler* bu tür sözleşmelerdir.
- her grup yazılımının son derece özel olduğunu düşünür ve yayınlanan arayüze bağlı kalmaya devam ettiği sürece herhangi bir zamanda değiştirme hakkını saklı tutar.

# Interface

- Interfaceden nesne üretilemez yani somutlaştırılamaz
  - Diğer sınıflar tarafında uygulanır. (`implements`)
  - Diğer interface ler tarafından genişletilebilir. (`extends`)

# Interface

- Erişim belirtici
  - Public – package private
- Extends
  - Diğer interfaceleri miras alabilir.
  - Sınıftan farklı olarak birde fazla miras mümkün
- Bütün metotlar public ve abstract dir
  - İstenirse yazılmayabilir
- Bütün değişkenler public static final dir
  - İstenirse yazılmayabilir.
- Gövdesi olan default metotlar olabilir
- Static metotlar olabilir
  - Static metot sınıfla ilişkiliydi nesne üretilmesine gerek yok
  - Zaten Interface den nesne üretilemez

```
public interface GroupedInterface extends Interface1,
Interface2, Interface3 {
    // constant declarations
    double E = 2.718282;
    // method signatures
    void doSomething (int i, double x);
    abstract int doSomethingElse(String s);
    default void someDefaultMethod() {
        System.out.println("This is the defa  t method");
    }
    static void someStaticMethod() {
        System.out.println("This is the  method");
    }
}
```

- Metot tanımında gövde olmadığına dikkat!
- Tanımdan sonra noktalı virgül ile bitiyor.

```java
public interface OperateCar {
   // constant declarations, if any

   // method signatures

   // An enum with values RIGHT, LEFT
   int turn(Direction direction,
            double radius,
            double startSpeed,
            double endSpeed);
   int changeLanes(Direction direction,
                   double startSpeed,
                   double endSpeed);
   int signalTurn(Direction direction,
                  boolean signalOn);
   int getRadarFront(double distanceToCar,
                     double speedOfCar);
   int getRadarRear(double distanceToCar,
                    double speedOfCar);
         ......
   // more method signatures
}
```

```java
public class OperateBMW760i implements OperateCar {

    // the OperateCar method signatures, with implementation
    // for example:
    public int signalTurn(Direction direction,
                          boolean signalOn)
    {
     // code to turn BMW's LEFT turn indicator lights on
     // code to turn BMW's LEFT turn indicator lights off
     // code to turn BMW's RIGHT turn indicator lights on
     // code to turn BMW's RIGHT turn indicator lights off
    }

    // other members, as needed -- for example, helper classes not
    // visible to clients of the interface
}
```

# Interface kullanımına genel örnek

- API'ler ticari yazılım ürünlerinde de yaygındır.
- Genellikle bir şirket, başka bir şirketin kendi yazılım ürününde kullanmak istediği karmaşık yöntemleri içeren bir yazılım paketi satar.
- Bir örnek;
  - son kullanıcı grafik programları yapan şirketlere (müşteri) satılan bir dijital görüntü işleme yöntemleri paketi olabilir.
  - Görüntü işleme şirketi (geliştirici), sınıflarını, müşterilerine açık hale getirdiği bir arabirimi uygulamak için yazar.
  - Grafik şirketi (müşteri) daha sonra arayüzde tanımlanan imzaları ve dönüş türlerini kullanarak görüntü işleme yöntemlerini çağırır.
  - Görüntü işleme şirketinin API'si (müşterilerine) herkese açık hale getirilirken, API'nin uygulanması korunan bir sır olarak tutulur.

# Veri Tipi Tanımlama

- Yeni bir arayüz tanımladığınızda, yeni bir referans veri tipi tanımlıyorsunuz.
- Arayüz adlarını başka herhangi bir veri türü adını kullanabileceğiniz her yerde kullanabilirsiniz.
- Türü interface olan bir başvuru değişkeni tanımlarsanız, ona atadığınız herhangi bir nesne interfaceyi uygulayan bir sınıfın örneği *olmalıdır* .

```java
public class Rectangle implements Relatable {
    public int width = 0;
    public int height = 0;
    public Point origin;

    public Rectangle () {
        origin = new Point(0, 0);
    }

    public void move(int x, int y) {
        origin.x = x;
        origin.y = y;
    }

    public int getArea() {
        return width * height;
    }
    public int isLargerThan(Relatable other) {
        RectanglePlus otherRect = (RectanglePlus)other;
        if (this.getArea() < otherRect.getArea())
            return -1;
        else if (this.getArea() > otherRect.getArea())
            return 1;
        else
            return 0;
    }
}
```

```java
public interface Relatable {
    public int isLargerThan(Relatable other);
}
```

- Relatable türünde değişken alıyor
- Relatable implement eden başka bir sınıftan türeyen nesne gönderilecek.
- Parametre atanırken Upcasting yapılıyor.
- Metot ilk satırında ise downcasting yapılarak getArea metoduna ulaşım sağlanmış oluyor.

- Relatable sınıfını uygulayan bir sınıftan ya da bu sınıfı miras almış herhangi bir sınıftan nesne parametre olabilir.

# Type casting

```
public Object findLargest(Object object1, Object object2) {
       Relatable obj1 = (Relatable)object1;
       Relatable obj2 = (Relatable)object2;
       if (obj1.isLargerThan(obj2) > 0)
          return object1;
       else
          return object2;
    }
```

- Object1 Bir Relatable türe dönüştürerek isLargerThan yöntemi çağırabilir

# Interface Geliştirilmek İstenirse ???

```
public interface Relatable {
     public int isLargerThan(Relatable other);
     public int isHeavierThan(Relatable other);
}
```

Yeni bir metot eklenmek istenirse ne olacak ???

! Reletable' ı uygulayan bütün sınıflara bu metodu ekleyerek güncellemek gerekecek

YA DA

- Eski interface'yi miras alan yeni bir interface yaz

```
public interface Comperable extends Relatable{
     public int isHeavierThan(Relatable other);
}
```

- Alternatif olarak yeni metodu default metot olarak ekle

```
public interface Relatable {
     public int isLargerThan(Relatable other);
     public default int isHeavierThan(Relatable other){
          //Birşeyler Yap
     }
}
```

# Default Metot Bulunan Interfaceyi Miras Alma

```
public interface Relatable {
        public int isLargerThan(Relatable other);
        public default int isHeavierThan(Relatable other){
                //Bir şeyler Yap
        }
}
```

1. Genişletilmiş arabiriminizin default metot ile ilgili bir şey yapmaya gerek yok.
   - Miras kuralları işler

```
public interface SecondI extends Relatable{
        void birseylerYap();
}
```

2. Default metodu yeniden abstract olarak tanımlayın.
   - Yeni interface yi uygulayan sınıf bu metodu uygulamalı

```
public interface SecondI extends Relatable{
        void birseylerYap();
        int isHeavierThan(Relatable other);
}
```

3. Default metodu yeniden tanımlayın tanımlayın (Override)
   - Bu interfaceyi uygulayan sınıf için metodun yeni hali geçerlidir.

```
public interface SecondI extends Relatable{
        void birseylerYap();
        public default int isHeavierThan(Relatable other){
         // Başka bir şeyler Yap
        }
}
```

# Default metot isimlerinde çakışma varsa ?

```
public class Horse {
    public String identifyMyself() {
        return "I am a horse.";
    }
}
public interface Flyer {
    default public String identifyMyself() {
        return "I am able to fly.";
    }
}
public interface Mythical {
    default public String identifyMyself() {
        return "I am a mythical creature.";
    }
}
public class Pegasus extends Horse implements Flyer,
Mythical {
    public static void main(String... args) {
        Pegasus myApp = new Pegasus();
        System.out.println(myApp.identifyMyself());
    }
}
```

<u>Ekran çıktısı ne olur ?</u>

I am a horse.

# Default metot isimlerinde çakışma varsa ?

```
public interface Animal {
    default public String identifyMyself() {
        return "I am an animal.";
    }
}
public interface EggLayer extends Animal {
    default public String identifyMyself() {
        return "I am able to lay eggs.";
    }
}
public interface FireBreather extends Animal { }
public class Dragon implements EggLayer, FireBreather {
    public static void main (String... args) {
        Dragon myApp = new Dragon();
        System.out.println(myApp.identifyMyself());
    }
}
```

Ekran çıktısı ne olur ?

I am able to lay eggs.

# Default metot isimlerinde çakışma varsa ?

```
public interface OperateCar {
    // ...
    default public int startEngine() {
        // Implementation
    }
}
public interface FlyCar {
    // ...
    default public int startEngine() {
        // Implementation
    }
}
public class FlyingCar implements OperateCar, FlyCar {
    public static void main (String... args) {
        FlyingCar myApp = new FlyingCar();
        System.out.println(myApp.startEngine());
    }

}
```

Ekran çıktısı ne olur ?

Derleme Hatası Verir!

FlayingCar sınıfı startEngine metodunu override etmeli

# Üst sınıf metodu abtract metodu uygulamış olur mu?

```
public interface Mammal {
    String identifyMyself();
}
public class Horse {
    public String identifyMyself() {
        return "I am a horse.";
    }
}
public class Mustang extends Horse implements Mammal {
    public static void main(String... args) {
        Mustang myApp = new Mustang();
        System.out.println(myApp.identifyMyself());
    }
}
```

Ekran çıktısı ne olur ?

I am a horse

Super sınıf identifyMyself metodunu override eder

# Anonim Sınıflar

- Aynı anda bir sınıfı bildirmenizi ve başlatmanızı sağlarlar.
- İsimleri olmaması dışında yerel sınıflar gibidirler.
- Yerel bir sınıfı yalnızca bir kez kullanmanız gerekiyorsa anonim sınıf kullanın.
- <u>Anonim sınıf içinde yapıcı metot (constructor) tanımlanamaz.</u>

```java
public class HelloWorldAnonymousClasses {
    interface HelloWorld {
        public void greet();
        public void greetSomeone(String someone);
    }
    public void sayHello() {
        class EnglishGreeting implements HelloWorld {
            String name = "world";
            public void greet() {
                greetSomeone("world");
            }
            public void greetSomeone(String someone) {
                name = someone;
                System.out.println("Hello " + name);
            }
        }
        HelloWorld englishGreeting = new EnglishGreeting();
        HelloWorld frenchGreeting = new HelloWorld() {
            String name = "tout le monde";
            public void greet() {
                greetSomeone("tout le monde");
            }
            public void greetSomeone(String someone) {
                name = someone;
                System.out.println("Salut " + name);
            }
        };
        englishGreeting.greet();
        frenchGreeting.greetSomeone("Fred");
    }
}
```

```java
public static void main(String... args) {
    HelloWorldAnonymousClasses myApp =
        new HelloWorldAnonymousClasses();
    myApp.sayHello();
}
```

Yerel Sınıf

Anonim Sınıf

# Lambda ifadeleri

Gerekli olanlar :

1. Tek metotlu interface (fonksiyonel interface)
2. Ananoim sınıftaki gibi "= new" ifadesi yerine " ()->" ifadesi kullanarak aslında anonim bir sınıf oluşturulmakta.

```
public class Calculator {

    interface IntegerMath {
        int operation(int a, int b);
    }

    public int operateBinary(int a, int b, IntegerMath op) {
        return op.operation(a, b);
    }

    public static void main(String... args) {

        Calculator myApp = new Calculator();
        IntegerMath addition = (a, b) -> a + b;
        IntegerMath subtraction = (a, b) -> a - b;
        System.out.println("40 + 2 = " + myApp.operateBinary(40, 2, addition));
        System.out.println("20 - 10 = " + myApp.operateBinary(20, 10, subtraction));
    }
}
```

# Abstract Sınıflar

- Kullanım amacı interface ye benzer
- İçeriğinde
  - Soyut metot içerebilir (gövdesi yoktur)
  - Somut metot içerebilir (gövdesi vardır)
    - public, protected, private olabilir.
  - Static ve final olmayan alanlar tanımlanabilir.
- Soyut sınıflar somutlaştırılamaz (nesne türetilemez),
- Ancak miras alınabilirler.
  - Bu yüzden bir sınıf sadece bir abstract sınıfı miras alabilir

- abstract sınıf:
  - Kodu, birbiriyle yakından ilişkili birkaç sınıf arasında paylaşmak istiyorsanız.
  - Soyut sınıfınızı genişleten sınıfların birçok ortak yönteme veya alana sahip olmasını veya belirleyici gerektirmesini beklerseniz.
  - Statik olmayan veya final olmayan alanlar bildirmek istiyorsunuz. Bu, ait oldukları nesnenin durumuna erişebilen ve durumunu değiştirebilen yöntemler tanımlamanıza olanak tanır.
- interface :
  - İlişkisiz sınıfların interface yi uygulamasını beklerseniz. Örneğin Cloneable bir çok ilgisiz sınıf tarafından uygulanır.
  - Belirli bir veri türünün davranışını belirtmek istiyorsunuz, ancak davranışını kimin uyguladığıyla ilgilenmiyorsanız.
  - Birden çok tür mirasından yararlanmak istiyorsanız.

# Jenerik Sınıflar

```
public class Box<T> {// T herhangi bir tip belirtir
    private T t;

    public void set(T t) { this.t = t; }
    public T get() { return t; }
}
Box<Integer> integerBox = new Box<Integer>();
```

- jenerikler, sınıfları, arabirimleri ve yöntemleri tanımlarken *türlerin* (sınıflar ve arabirimler) parametre olmasını sağlar.
- Metot parametresinden farkı
  - Metot parametrelerinin içeriğinin <u>değer</u> olması,
  - Jenerik parametrelerinin içeriğinin <u>tür</u> olması

```
public class Box {
    private Object object;

    public void set(Object object) { this.object = object; }
    public Object get() { return object; }
}


Box integerBox = new Box(new Integer());
```

# Jenerik Sınıflar

- Birden fazla parametre almak da mümkün

```
public interface Pair<K, V> {
    public K getKey();
    public V getValue();
}

public class OrderedPair<K, V> implements Pair<K, V> {

    private K key;
    private V value;

    public OrderedPair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey(){ return key; }
    public V getValue() { return value; }
}
```

```
Pair<String, Integer> p1 = new OrderedPair<String, Integer>("Even", 8);
Pair<String, String>  p2 = new OrderedPair<String, String>("hello", "world");
OrderedPair<String, Box<Integer>> p = new OrderedPair<>("primes", new Box<Integer>());
```

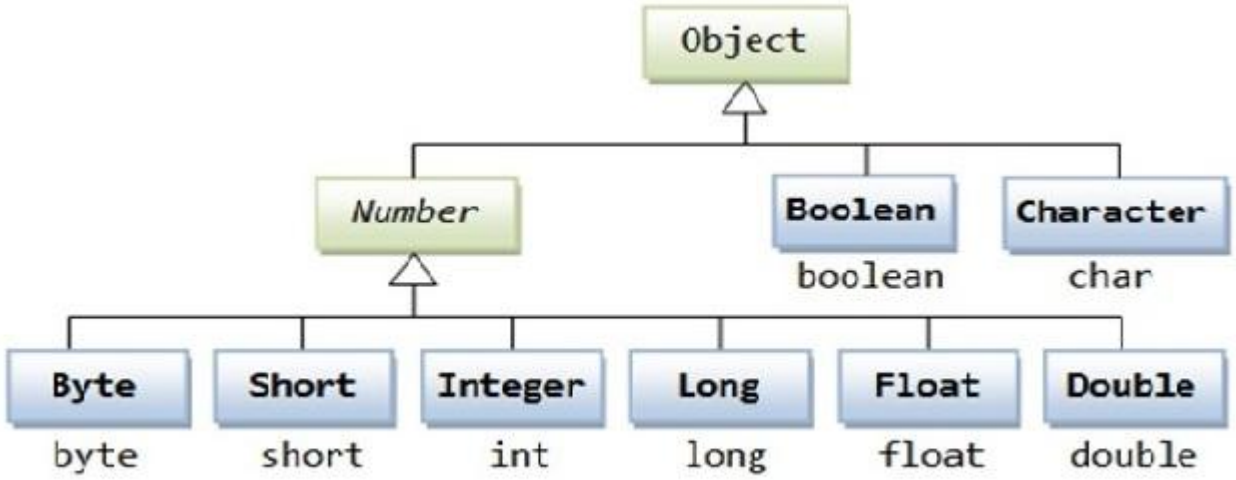
```
Object someObject = new Object();
Integer someInteger = new Integer(10);
someObject = someInteger;
```

Burada Bir Problem Var mı ?

Peki ya burada ?

```
public void someMethod(Number n) { /* ... */ }

someMethod(new Integer(10));
someMethod(new Double(10.1));
```




```
Box<Number> box = new Box<Number>();
box.set(new Integer(10));
box.set(new Double(10.1));
```

```
Box<Number> box = new Box();
Box<Integer> box2 = new Box();
box.boxTest(box2);
```

Ne Değişti ?

**Number t** = new **Integer(10)**

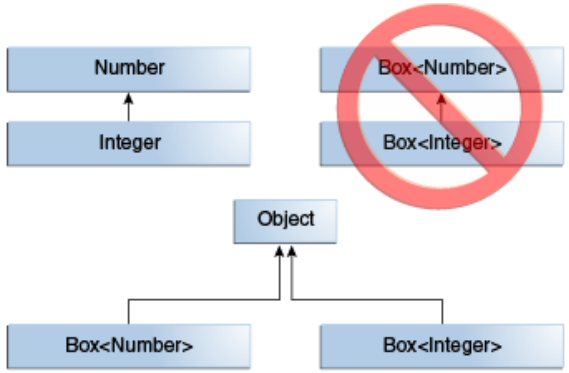
Peki ya şimdi ?

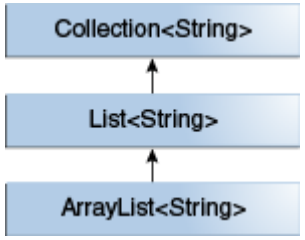
**Box<Number> t = Box2**

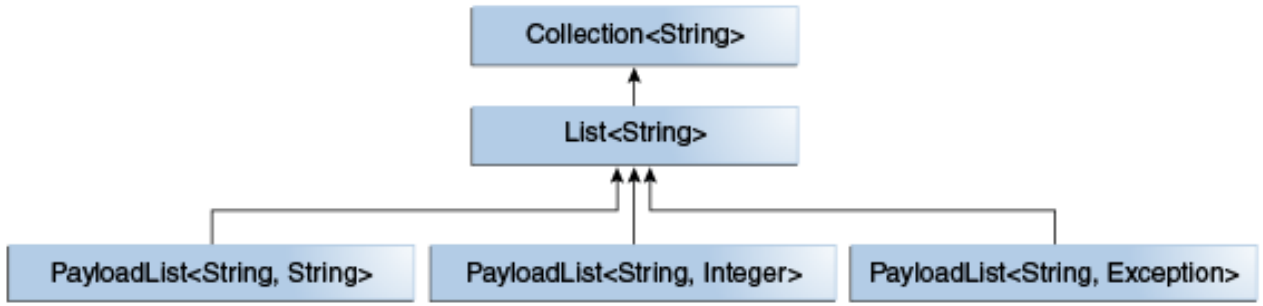
**(not: box2 Box<Integer> türünde)**

```
public class Box<T> {
    // T stands for "Type"
    private T t;

    public void set(T t) { this.t = t; }
    public T get() { return t; }
}   public void boxTest(Box<T> n) {
/* ... */ }
}
```






```
interface PayloadList<E,P> extends List<E> {
  void setPayload(int index, P val);
  ...
}

class PList<E,P> implements PayloadList<E,P>
{
    ...
}
```

```
PayloadList<String,Integer> pl1 = new PList <String,Integer>();
PayloadList<Integer,Integer> pl2 = new PList <Integer,Integer>();
pl1=pl2;
```

`pl1=pl2;` satırı hata verecektir.

Çünkü

`PayloadList<String,Integer>` ile `PayloadList<Integer,Integer>`

Aynı sınıf değiller

# Jenerik Metotlar

```
public class Util {
    public static <K, V> boolean compare(Pair<K, V> p1, Pair<K, V> p2) {
        return p1.getKey().equals(p2.getKey()) &&
               p1.getValue().equals(p2.getValue());
    }
}

public class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }
    public void setKey(K key) { this.key = key; }
    public void setValue(V value) { this.value = value; }
    public K getKey()   { return key; }
    public V getValue() { return value; }
}
```

```
Pair<Integer, String> p1 = new Pair<>(1, "apple");
Pair<Integer, String> p2 = new Pair<>(2, "pear");
boolean same = Util.<Integer, String>compare(p1, p2);
```

=

```
Pair<Integer, String> p1 = new Pair<>(1, "apple");
Pair<Integer, String> p2 = new Pair<>(2, "pear");
boolean same = Util.compare(p1, p2);
```

# Parametrelerdeki miras hiyerarşisi kullanılmak isteniyorsa?

## ÇÖZÜM

### Joker Karakter

### <?>

```
List<Integer> li = Arrays.asList(1, 2, 3);
List<String>  ls = Arrays.asList("one", "two", "three");
printList(li);
printList(ls);
```

```
public static void printList(List<?> list) {
    for (Object elem: list)
        System.out.print(elem + " ");
    System.out.println();
}
```

✓

```
public static void printList(List<Object> list) {
    for (Object elem : list)
        System.out.println(elem + " ");
    System.out.println();
}
```

✗

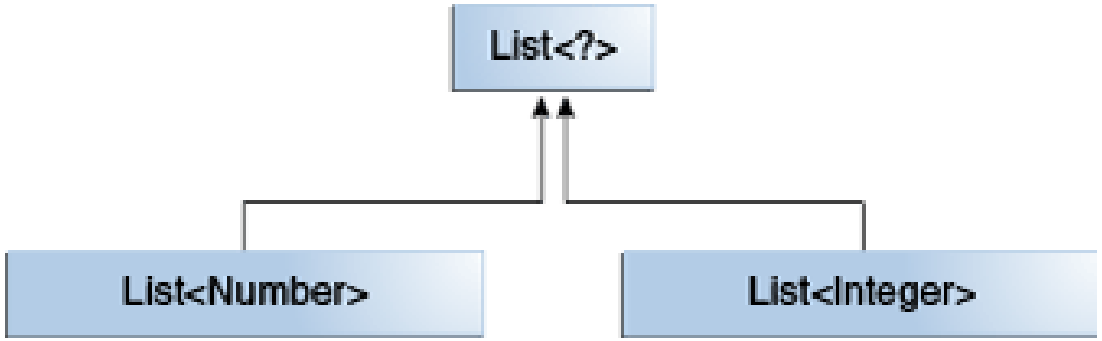
# Üst Sınır Belirleme

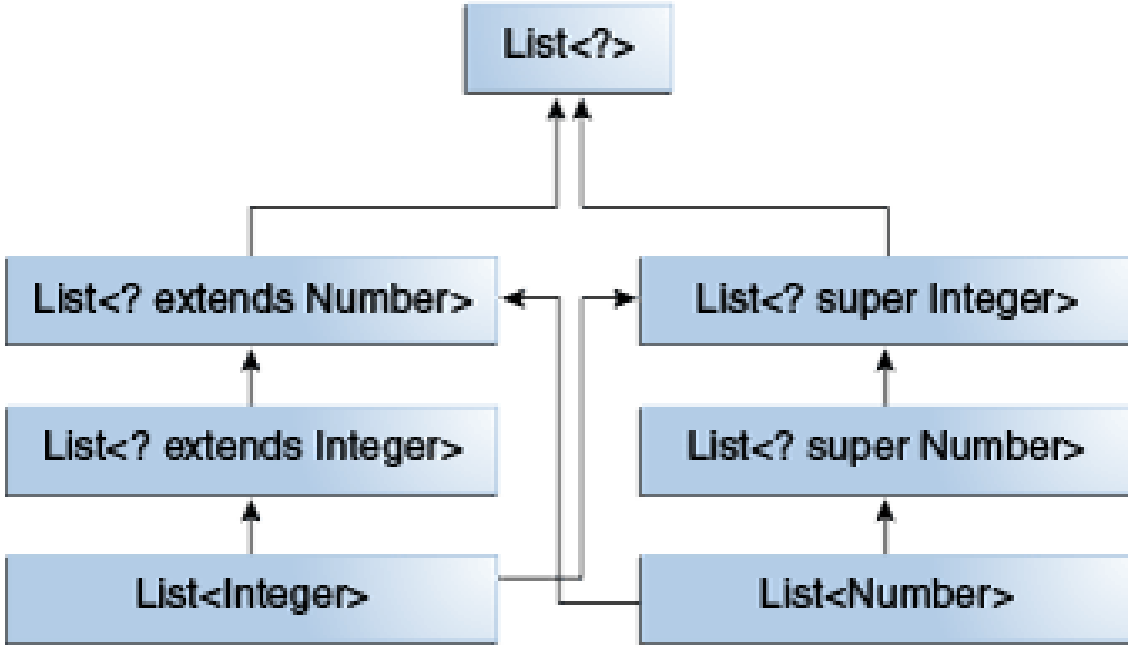
```
abstract class Sekil {
    public abstract void ciz(Tuval c);
}
public class Daire extends Sekil {
    private int x, y, cap;
    public void ciz(Tuval c) {
        ...
    }
}
public class Dikdortgen extends Sekil {
    private int x, y, en, boy;
    public void draw(Tuval c) {
        ...
    }
}
public class Tuval {
    public void ciz(Sekil s) {
        s.ciz(this);
    }
    public void hepsiniCiz(List<? extends Sekil> sekiller) {
        for (Sekil s: sekiller) {
            s.ciz(this);
        }
    }
}
```

# Alt Sınır Belirleme

```
abstract class Sekil {
    public abstract void ciz(Tuval c);
}
public class Daire extends Sekil {
    private int x, y, cap;
    public void ciz(Tuval c) {
        ...
    }
}
public class Dikdortgen extends Sekil {
    private int x, y, en, boy;
    public void draw(Tuval c) {
        ...
    }
}
public class Tuval {
    public void ciz(Sekil s) {
        s.ciz(this);
    }
     public void hepsiniCiz(List<? super Daire> sekiller) {
        for (Sekil s: sekiller) {
            s.ciz(this);
        }
    }
}
```

Dikdörgen sınıfı dahil olmuyor.


List<?>
List<Number>
List<Integer>


List<?>
List<? extends Number>
List<? super Integer>
List<? extends Integer>
List<? super Number>
List<Integer>
List<Number>

# KAYNAK

- https://docs.oracle.com/javase/tutorial/index.html